ANÁLISIS DEL RETO

Christian Bravo, 202322012, ca.bravom@uniandes.edu.co

Nicolás Fajardo, 202413083, n.fajardoa@uniandes.edu.co

Dennys Lozano, 202413859, da.lozanom1@uniandes.edu.co

Carga de datos y tablas de hash

Asignación de requerimientos: el Requerimiento 1 lo desarrolló Christian Bravo, el Requerimiento 2 lo desarrolló Nicolás Fajardo y el Requerimiento 3 lo desarrolló Dennys Lozano. La carga de datos y los Requerimientos 4, 5 y 6 son grupales.

En la carga se construyen solo tres tablas de hash y cada una la usan varios requerimientos. La idea es no crear estructuras dentro de cada requerimiento a partir de la lista principal y tampoco construir en la carga una tabla que sirva para una sola consulta. Para lograrlo se usan mapas anidados: una tabla externa cuyo valor es otra tabla de hash interna.

Tabla por_mes. Tipo Separate Chaining. Llave: los primeros siete caracteres de Order_Date, que son el año y el mes del pedido. Valor: una tabla interna por Linear Probing cuya llave es la fecha completa del pedido y cuyo valor es una single linked list con los pedidos de ese día. Propósito: el Requerimiento 1 toma el mes completo y el Requerimiento 3 llega directo a un día.

Tabla por_pais. Tipo Separate Chaining. Llave: Country. Valor: una tabla interna por Linear Probing cuya llave es Product y cuyo valor es una single linked list con los pedidos de ese país y ese producto. Propósito: los Requerimientos 2 y 5 toman todos los productos de un país, el Requerimiento 4 llega directo a la combinación producto y país, y el Requerimiento 6 recorre los cinco países y en cada uno pide un solo producto.

Tabla channel_count. Tipo Linear Probing. Llave: Channel. Valor: número de pedidos del canal. Propósito: reporte de la carga.

Justificación del tipo de tabla. Las tablas externas son Separate Chaining porque el número de meses y de países depende del archivo y puede crecer con los datos; esta técnica acepta un factor de carga mayor a 1 (se usa 4) sin llenarse y el costo promedio de get y put sigue siendo constante. Las tablas internas y la de canales son Linear Probing porque el número de llaves es pequeño y acotado (máximo 31 días en un mes, 12 productos y 3 canales); con factor de carga 0.5 los clusters son cortos, no hay eliminaciones y no se crean nodos extra, por lo que la búsqueda es más rápida y ocupa menos memoria.

Justificación de las listas. Los valores de las tablas internas son single linked list porque solo se agregan pedidos al final (add_last en O(1)) y luego se recorren nodo por nodo, nunca por posición. La lista principal y las listas de resultados son array_list porque merge sort necesita acceder por posición en O(1).

Descripción

Se lee el archivo una sola vez. Por cada fila se crea el pedido y se agrega al final de la lista principal. En el mismo recorrido se inserta el pedido en por_mes (primero el mes y luego el día) y en por_pais (primero el país y luego el producto), se suma uno al contador del canal y se actualizan el pedido de menor y mayor Amount y las fechas extremas. Al terminar, el reporte copia la lista principal, la ordena con merge sort descendente por Amount y toma los primeros cinco y los últimos cinco.

Entrada	Nombre del archivo CSV que elige el usuario.
Salidas	Catálogo con la lista principal de pedidos y las tres tablas de hash. Reporte con tiempo de carga, total de pedidos, pedidos por canal, fecha más antigua y más reciente, pedido con menor y mayor Amount, y los primeros cinco y últimos cinco pedidos ordenados descendente por Amount con desempate por Order_ID ascendente.
Implementado (Sí/No)	Sí. Implementado por Christian Bravo y ajustado por el grupo.

Análisis de complejidad

Sea n el número total de pedidos. Cada operación de las tablas de hash es O(1) en promedio, así que la lectura con todos los índices es O(n). El paso dominante es el merge sort del reporte, que es O(n log n). El algoritmo de ordenamiento usado es merge sort porque es estable y siempre O(n log n), sin importar cómo vengan los datos.

Pasos	**Complejidad**
Leer cada fila del CSV y crear el pedido	O(n)
Agregar el pedido a la lista principal (add_last en array_list)	O(n)
Insertar en por_mes: get y put externo, get y put interno, add_last en la single linked list	O(n)
Insertar en por_pais con los mismos pasos	O(n)
Contar el canal en channel_count	O(n)
Actualizar menor y mayor Amount y fechas extremas	O(n)
Reporte: copiar la lista y ordenarla con merge sort	O(n log n)
Tomar los primeros y últimos cinco y armar la tabla de canales	O(1)
TOTAL	O(n log n)

Análisis

Con el archivo del 100 por ciento (193608 pedidos) la lectura e indexación tomó aprox 2146 ms en nuestras pruebas. Una versión anterior con seis tablas (una por cada consulta) tardaba aprox 5000 ms; al unificar en tres tablas anidadas la carga se redujo a menos de la mitad del tiempo y todas las consultas siguen accediendo directo por llave.

Requerimiento 1

Consultar pedidos por mes específico y rango de descuento. Tablas usadas: por_mes (Separate Chaining) y su tabla interna de días (Linear Probing).

Descripción

Se arma la llave del mes a partir del año y el mes, se hace get en por_mes y se obtiene la tabla de días de ese mes. Con value_set se sacan las listas de cada día y se recorre cada una nodo por nodo. Los pedidos con descuento dentro del rango (incluye los extremos) se agregan a un array_list y se acumulan sus sumas. Al final se ordena con merge sort y se calculan los promedios.

Entrada	Catálogo, año, mes, descuento mínimo y descuento máximo.
Salidas	Tiempo de ejecución, número de pedidos, monto total recaudado, promedio de Price_per_Box y de Boxes_Shipped, y la lista de pedidos ordenada descendente por Discount_Pct, con desempate por Amount descendente y Order_ID ascendente.
Implementado (Sí/No)	Sí. Implementado por Christian Bravo.

Análisis de complejidad

m es el número de pedidos del mes y k los que pasan el filtro, con k menor o igual a m. Gracias a la tabla solo se recorre el mes pedido y no la lista principal; con los datos del reto m es aprox n dividido entre 24. En el peor caso, si todos los pedidos fueran del mismo mes y pasaran el filtro, sería O(n log n). Algoritmo de ordenamiento: merge sort, O(k log k).

Pasos	**Complejidad**
Armar la llave y hacer get en por_mes	O(1)
value_set de la tabla de días (máximo 31 llaves)	O(1)
Recorrer los m pedidos del mes, filtrar y acumular	O(m)
Ordenar los k pedidos filtrados con merge sort	O(k log k)
Calcular promedios	O(1)
TOTAL	O(m + k log k)

Análisis

Con el archivo del 100 por ciento, la consulta de marzo de 2023 con descuento entre 10 y 20 devolvió 4123 pedidos en aprox 104 ms. La mayor parte del tiempo se va en el ordenamiento, porque el acceso al mes es directo.

Requerimiento 2

Consultar pedidos por país y rango de cajas enviadas. Tablas usadas: por_pais (Separate Chaining) y su tabla interna de productos (Linear Probing).

Descripción

Se hace get del país en por_pais y se obtiene su tabla de productos. Se recorre cada producto con key_set y su lista nodo por nodo. Los pedidos con cajas dentro del rango se agregan al resultado y se cuentan por producto mientras se recorre su lista, así que la tabla ya entrega los pedidos agrupados por producto y no hace falta otra estructura para contarlos. Con ese conteo se sabe cuántos productos diferentes hay y cuál es el más frecuente (en empate gana el de nombre alfabéticamente menor). Al final se ordena con merge sort.

Entrada	Catálogo, país, número mínimo y número máximo de cajas enviadas.
Salidas	Tiempo de ejecución, número de pedidos, número de productos diferentes, promedio de Marketing_Spend y de Amount, producto más frecuente con su número de pedidos, y la lista ordenada ascendente por Boxes_Shipped, con desempate por Order_Date descendente y Order_ID ascendente.
Implementado (Sí/No)	Sí. Implementado por Nicolás Fajardo.

Análisis de complejidad

m es el número de pedidos del país, aprox n dividido entre 5 con los datos del reto, y k los que pasan el filtro. P es una constante pequeña. En el peor caso, un solo país con todos los pedidos dentro del rango, sería O(n log n). Algoritmo de ordenamiento: merge sort, O(k log k).

Pasos	**Complejidad**
get del país en por_pais	O(1)
key_set de la tabla de productos (P productos, máximo 12)	O(P)
Recorrer los m pedidos del país, filtrar, acumular y contar por producto	O(m)
Elegir el producto más frecuente	O(P)
Ordenar los k pedidos con merge sort	O(k log k)
TOTAL	O(m + k log k)

Análisis

Con el archivo del 100 por ciento, Germany con cajas entre 100 y 200 devolvió 9106 pedidos en aprox 235 ms. Es el requerimiento con más resultados porque un país concentra muchos pedidos, y por eso el ordenamiento pesa más que el recorrido.

Requerimiento 3

Listar los pedidos en una fecha específica y rango de costo total. Tablas usadas: por_mes (Separate Chaining) y su tabla interna de días (Linear Probing).

Descripción

Con los primeros siete caracteres de la fecha se hace get del mes en por_mes y luego get de la fecha completa en la tabla de días, con lo que se llega directo a la lista de pedidos de ese día. Se recorre la lista nodo por nodo y los pedidos con Amount dentro del rango se agregan a un array_list mientras se suman el Amount, el Marketing_Spend y el aporte comercial (Amount menos Marketing_Spend). Al final se ordena con merge sort.

Entrada	Catálogo, fecha del pedido, valor mínimo y valor máximo de costo total (Amount).
Salidas	Tiempo de ejecución, número de pedidos, monto total recaudado, inversión total en mercadeo, aporte comercial total y la lista de pedidos ordenada descendente por Amount, con desempate por aporte comercial descendente y Order_ID ascendente.
Implementado (Sí/No)	Sí. Implementado por Dennys Lozano.

Análisis de complejidad

d es el número de pedidos del día y k los que pasan el filtro. Con 730 días en los datos, d es aprox n dividido entre 730, unos 265 pedidos con el archivo completo. En el peor caso, todos los pedidos en un mismo día, sería O(n log n). Algoritmo de ordenamiento: merge sort, O(k log k).

Pasos	**Complejidad**
get del mes en por_mes	O(1)
get del día en la tabla interna	O(1)
Recorrer los d pedidos del día, filtrar y acumular	O(d)
Ordenar los k pedidos con merge sort	O(k log k)
TOTAL	O(d + k log k)

Análisis

Con el archivo del 100 por ciento, el 28 de marzo de 2023 con Amount entre 100 y 2000 devolvió 299 pedidos en aprox 2.7 ms. Es de los más rápidos porque la tabla anidada deja el recorrido en un solo día, sin tocar el resto de los pedidos.

Requerimiento 4

Precio promedio para una combinación producto y país. Tablas usadas: por_pais (Separate Chaining) y su tabla interna de productos (Linear Probing).

Descripción

Se hace get del país en por_pais y luego get del producto en su tabla interna, con lo que se obtiene directo la lista de pedidos de la combinación. Se recorre nodo por nodo sumando para los promedios y manteniendo dos variables, top_1 y top_2, que se actualizan en la misma pasada con las reglas de desempate. No se necesita ordenar porque solo se piden los dos mayores.

Entrada	Catálogo, producto y país.

Salidas	Tiempo de ejecución, número de pedidos, promedio de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped, y los dos pedidos de mayor Amount (desempate por menor Marketing_Spend y luego Order_ID ascendente).
Implementado (Sí/No)	Sí. Requerimiento grupal, implementado por Christian Bravo, Nicolás Fajardo y Dennys Lozano.

Análisis de complejidad

q es el número de pedidos de la combinación producto y país. No se usa algoritmo de ordenamiento: mantener los dos mayores en un solo recorrido cuesta O(q), menos que ordenar, que costaría O(q log q). En el Reto 1 el mismo requerimiento copiaba toda la lista a una single linked list y la recorría completa, lo que era O(n) sin importar cuántos pedidos tuviera la combinación.

Pasos	**Complejidad**
get del país en por_pais	O(1)
get del producto en la tabla interna	O(1)
Recorrer los q pedidos de la combinación, sumar y actualizar top_1 y top_2	O(q)
Calcular promedios	O(1)
TOTAL	O(q)

Análisis

Comparación experimental con la consulta Truffle Gift Box en Japan, la misma en los dos retos, cinco ejecuciones por tamaño en el mismo computador y sin contar la carga ni la impresión. Los resultados completos y la gráfica están en el archivo ISIS1225 TablasDatosReto2 de esta carpeta.

Reto 1, tiempo promedio: 10 por ciento 11.0 ms, 20 por ciento 27.6 ms, 30 por ciento 41.1 ms, 40 por ciento 64.0 ms, 50 por ciento 84.4 ms, 60 por ciento 100.8 ms, 70 por ciento 161.4 ms, 80 por ciento 135.9 ms, 90 por ciento 145.3 ms y 100 por ciento 183.2 ms.

Reto 2, tiempo promedio: 10 por ciento 0.09 ms, 20 por ciento 0.21 ms, 30 por ciento 0.33 ms, 40 por ciento 0.43 ms, 50 por ciento 0.59 ms, 60 por ciento 0.79 ms, 70 por ciento 0.83 ms, 80 por ciento 0.95 ms, 90 por ciento 1.09 ms y 100 por ciento 1.22 ms.

Los pedidos encontrados fueron de 179 con el 10 por ciento a 1838 con el 100 por ciento. En el Reto 1 el tiempo crece con n porque se recorre toda la lista. En el Reto 2 también crece, pero solo porque crece q, la cantidad de pedidos de la combinación; con el archivo completo el Reto 2 es aprox 150 veces más rápido, lo que coincide con pasar de O(n) a O(q).

Requerimiento 5

Obtener los N productos con mayor recaudo para un país y un rango de fechas. Tablas usadas: por_pais (Separate Chaining) y su tabla interna de productos (Linear Probing).

Descripción

Se hace get del país en por_pais. Por cada producto de su tabla interna se recorre su lista nodo por nodo, se filtran los pedidos dentro del rango de fechas (incluye los extremos) y se acumulan en un registro de ese producto. Como la tabla ya separa los pedidos por producto, no hace falta otra tabla para agrupar. Los productos con al menos un pedido se agregan a un array_list, se ordenan con merge sort (recaudo descendente, cajas descendente y nombre ascendente) y se toman los primeros N.

Entrada	Catálogo, N, país, fecha inicial y fecha final.
Salidas	Tiempo de ejecución, número de pedidos filtrados, número de productos diferentes, recaudo total y el top N de productos con número de pedidos, cajas totales, recaudo, precio promedio, descuento promedio y Marketing_Spend total.
Implementado (Sí/No)	Sí. Implementado por Christian Bravo (requerimiento grupal).

Análisis de complejidad

m es el número de pedidos del país y P el número de productos, máximo 12, por lo que el ordenamiento es casi constante y domina el recorrido O(m). Algoritmo de ordenamiento: merge sort sobre P elementos.

Pasos	**Complejidad**
get del país en por_pais	O(1)
key_set de la tabla de productos	O(P)
Recorrer los m pedidos del país, filtrar por fecha y acumular por producto	O(m)
Calcular promedios por producto	O(P)
Ordenar los P productos con merge sort	O(P log P)
Tomar los primeros N	O(N)
TOTAL	O(m + P log P)

Análisis

Con el archivo del 100 por ciento, India entre el 1 de junio de 2022 y el 15 de febrero de 2023 con N igual a 5 tomó aprox 24 ms. El tiempo depende casi solo de cuántos pedidos tiene el país, porque lo que se ordena son a lo sumo 12 productos.

Requerimiento 6

Obtener los N países con mayor eficiencia de mercadeo para un producto y un rango de fechas. Tablas usadas: por_pais (Separate Chaining) y su tabla interna de productos (Linear Probing).

Descripción

Se recorren los países con key_set de por_pais y en cada uno se hace get del producto en su tabla interna. Así se llega solo a los pedidos de ese producto, ya separados por país. Cada lista se recorre nodo por nodo filtrando por fechas y acumulando en un registro del país. Los países con Marketing_Spend total mayor a cero calculan su eficiencia (suma de Amount dividida por suma de Marketing_Spend) y

entran a un array_list, que se ordena con merge sort (eficiencia descendente, Amount total descendente y Country ascendente). Se toman los primeros N.

Entrada	Catálogo, N, producto, fecha inicial y fecha final.
Salidas	Tiempo de ejecución, número de pedidos filtrados, número de países diferentes, Amount total, Marketing_Spend total y el top N de países con número de pedidos, Amount total, Marketing_Spend total, cajas totales, precio promedio y eficiencia de mercadeo.
Implementado (Sí/No)	Sí. Implementado por Nicolás Fajardo (requerimiento grupal).

Análisis de complejidad

p es el número de pedidos del producto en todos los países, aprox n dividido entre 12. C es muy pequeño, así que domina el recorrido O(p). Algoritmo de ordenamiento: merge sort sobre C elementos.

Pasos	**Complejidad**
key_set de por_pais (C países, 5 en los datos)	O(C)
get del producto en la tabla interna de cada país	O(C)
Recorrer los p pedidos del producto, filtrar por fecha y acumular por país	O(p)
Calcular la eficiencia y armar el ranking	O(C)
Ordenar los países con merge sort	O(C log C)
Tomar los primeros N	O(N)
TOTAL	O(p + C log C)

Análisis

Con el archivo del 100 por ciento, 70% Dark Bar durante 2022 con N igual a 3 tomó aprox 91 ms. Se recorre solo el producto pedido y no toda la lista principal, y la agrupación por país sale directo de la estructura de la carga.